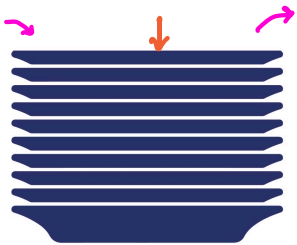


Agenda:

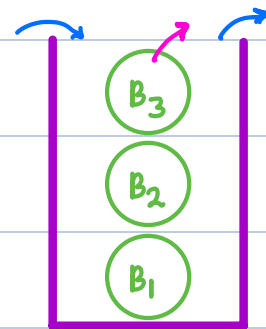
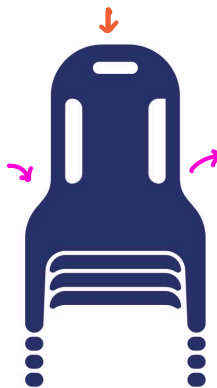
1. Introduction
 2. Operations of Stack
 3. Undo & Redo
 4. Stack Implementation via Dynamic Arrays
 5. Stack Implementation via Linked List
 6. Balanced Parenthesis
 7. Evaluate Postfix Expression
-

Introduction

1. Stack of Plates



2. Stack of Chairs



A stack is a **linear data structure** that stores information in a sequence, from **bottom to top**.

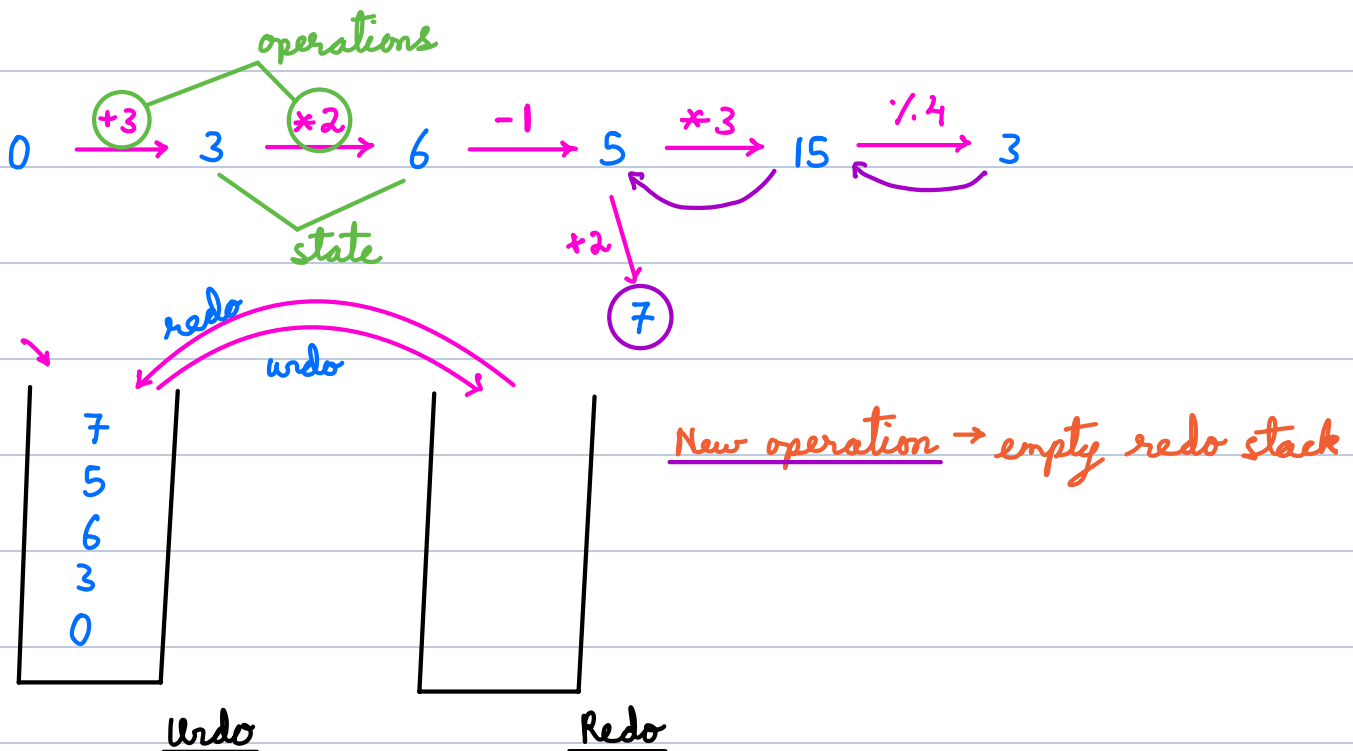
The data items can **only be accessed from the top** and new elements can only be added to the top, i.e it follows **LIFO (Last In First Out)** principle.

Operation on Stack

1. `push(X)` - insert element X from top of stack
 2. `pop()` - remove top element from stack
 3. `top()` / `peek()` - get top element from stack
 4. `isEmpty()` - check if stack is empty
- } $T.C = O(1)$

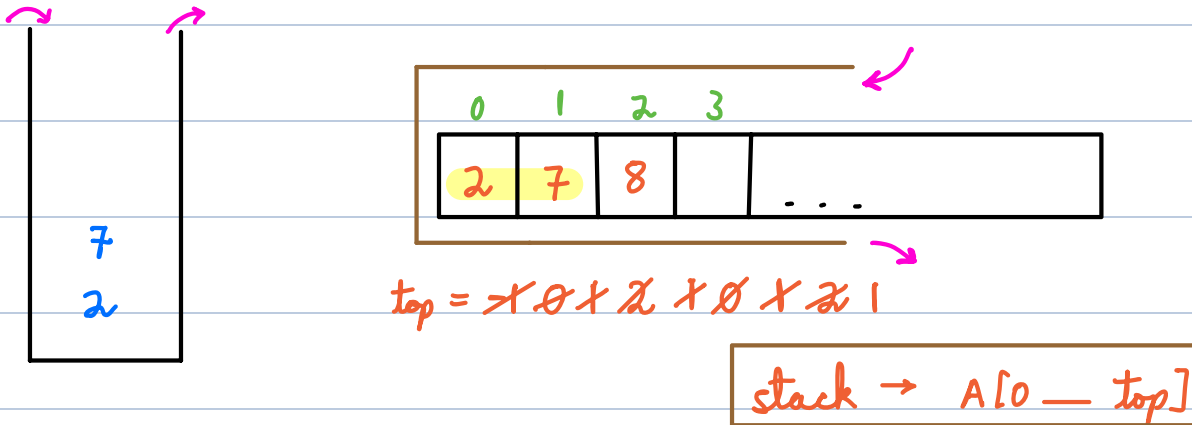
Undo-Redo Using Stacks

- **Undo()** → Reverse the most recent editing action (remove last change)
- **Redo()** → Bring back an action that was undone
- Found in text editors, IDEs, browsers, Photoshop, etc.



Q1: Implement Stack using Arrays

Example: [✓]push(2) [✓]push(3) [✓]push(5) ⁵peek() ⁵pop()
 ³pop() ^{false}isEmpty() [✓]push(7) [✓]push(8) ⁸pop() ⁷peek()



top = -1

```
boolean isEmpty() {  
    return (top == -1)  
}
```

```
void push(x) {  
    top++  
    A[top] = x  
}
```

// overflow -> use arraylist ✓

// underflow -> check for empty stack ✓

```
int peek() {  
    if (isEmpty())  
        return -1  
    return A[top]  
}
```

```
int pop() {  
    if (isEmpty())  
        return -1
```

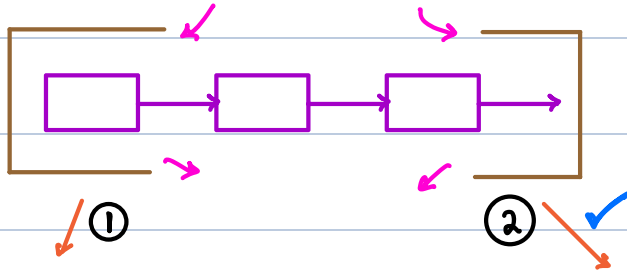
top--

return A[top+1]

```
}
```

Q2: Implementation of Stack using Linked List

Example: [✓]push(2) [✓]push(3) [✓]push(5) ⁵peek() ⁵pop()
 ³pop() ^{false}isEmpty() [✓]push(7) [✓]push(8) ⁸pop() ⁷peek()

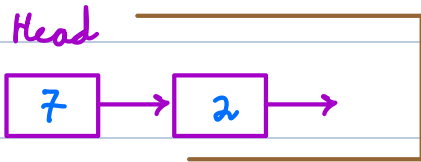


push → insert as last node ✓

pop → remove last node ✗

push → insert as first node ✓

pop → remove first node ✓



1) push(x) → insert as Head

2) isEmpty() → (Head == null)

3) peek() → Head.data

4) pop() → top = Head.data Head = Head.next

} TC = O(1)

Q3: Check whether the given sequence of parenthesis is valid ?

Example:

i/p $\rightarrow$ () { } []

a. [() { }] ✓

b. { () } ✗

c. [() { }] () { } ✓

d. ({ } [()) ✗

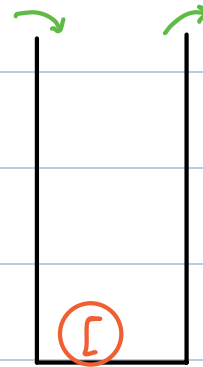
open without close

} _____ ✗

()] ✗

close without open

[([] { } ()) [] }



store open brackets $\rightarrow$ SC ✓

latest opening bracket should match with closing bracket $\rightarrow$ stack

for $i \rightarrow 0$ to $(N-1)$ {

if ($s[i] == '('$ || $s[i] == '{'$ || $s[i] == '['$)

st.push(s[i])

else if ($s[i] == ')'$) {

if (!st.isEmpty() && st.peek() == '(') st.pop()

else return false

} else if ($s[i] == '}'$)

if (!st.isEmpty() && st.peek() == '{') st.pop()

```

        else return false
    } else { // '['
        if (!st.isEmpty() && st.peek() == '[') st.pop()
        else return false
    }
}

return st.isEmpty()

```

$TC = \underline{O(N)}$ $SC = \underline{O(N)}$

Infix

2 + 5

3 + (2 * 6)

(3 + 2) * 6

Postfix

2 5 +

3 2 6 * +

3 2 + 6 *

operand1 operator operand2

operand1 operand2 operator

Q4: Evaluate the given valid postfix expression.

Example:

$s[i] \rightarrow \{0, 1 \dots 9, +, -, *, /\}$

a. 5 3 - $5 - 3 = \underline{2}$

b. 2 3 5 * + $2 + (3 * 5) = \underline{17}$

c. 3 5 + 2 - 2 5 * -

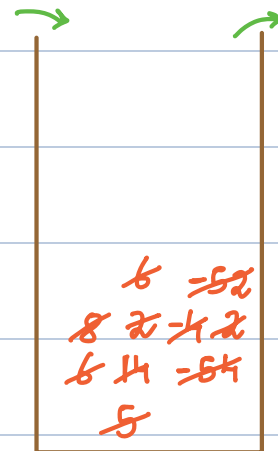
$6 - 10 = \underline{\underline{-4}}$

✓ ✓ ✓ ↓ ✓ ✓ ↓ ↓ ✓ ↓ ↓
5 6 8 + 2 6 - * 2 + -

t2 operator t1

t1 = 8 t2 = 6

$6 + 8 = 14$



t1 = 6 t2 = 2 $2 - 6 = -4$

t1 = -4 t2 = 14 $14 * (-4) = -56$

t1 = 2 t2 = -54 $-54 + 2 = -52$

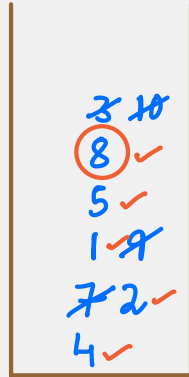
t1 = -52 t2 = 5 $5 - (-52) = \underline{\underline{57}}$ (Ans)

TC = O(N)

SC = O(N)

Consider the following stack operations (initially stack is empty):

push(4); ✓
push(7); ✓
pop(); ✓
push(2); ✓
push(9); ✓
pop(); ✓
push(1); ✓
push(5); ✓
push(8); ✓
push(3); ✓
pop(); ✓
push(10); ✓
pop(); ✓



What will be the top element of the stack (most recently inserted element that has not been removed yet) ? and What is the size of the stack at the end?

Evaluate following postfix operations:

8 3 2 * 5 + 7 2 - * 4 6 * + 9 - -

